

Manual Online - Valley Hybrid DB Bootstrap

Fonte viva em Markdown com vertente PDF derivada para schema PostgreSQL + MongoDB.

\n\n# Fonte: MANUAL_ONLINE\README.md\n\n# Manual Online - Valley Hybrid DB Bootstrap

Este manual e a fonte viva da documentacao tecnica do banco hibrido Valley/Nexora.

Ele deve ser atualizado sempre que qualquer schema, regra de banco, indice, trigger, validator ou script operacional for alterado.

O PDF em `output/pdf/VALLEY\_MANUAL\_ONLINE.pdf` e a versao derivada deste Markdown para leitura executiva e compartilhamento.

Fonte de Verdade

A regra principal deste projeto e o `AGENTS` recebido nesta worktree Valley.

Os PDFs enviados servem como referencia semantica de produto, mas nao substituem o fluxo local de tres passos.

A estrategia adotada e `core-first`: primeiro blindar identidade, wallets, ledger, orders e NoSQL operacional.

Nao foi criada compatibilidade com `platform.users` ou multi-schema legado nesta entrega inicial.

Arquivos Gerados

`database/postgres/001\_core\_identity\_wallets.sql` cria o nucleo relacional de identidade e carteiras.

Esse arquivo define `users`, `pj\_profiles`, `rider\_profiles`, `wallets` e `led\_cards`.

Ele tambem cria enums, chaves estrangeiras, indices, checks e triggers para coerencia de perfil e posse de carteira/cartao.

`database/postgres/002\_financial\_ledger\_equity\_orders.sql` cria o motor transacional.

Esse arquivo define `orders`, `transactions` e `equity\_ledger`.

Ele tambem cria os enums de transacao, pedidos e equity, alem de triggers append-only para impedir `UPDATE` e `DELETE` em livros financeiros.

`database/mongodb/001\_ai\_social\_telemetry.mongo.js` cria ou atualiza as colecoes NoSQL.

Esse arquivo define `ai\_memory`, `social\_videos`, `influencer\_metrics` e `telemetry\_logs` com JSON Schema Validation.

Ele tambem cria indices para IA, feed social, analytics de influenciador e telemetria geoespacial.

`database/postgres/003\_database\_comments\_ptbr.sql` cria a camada de comentarios nativos do PostgreSQL.

Esse arquivo documenta tipos, tabelas, colunas, funcoes e triggers em portugues simples com termos tecnicos em ingles.

Ele deve ser executado depois dos scripts `001` e `002`.

`database/postgres/004\_v47\_control\_plane\_modules\_rules.sql` implanta o plano de controle v47 viavel.

Esse arquivo cria catalogo de modulos, usuarios admin, permissoes, regras versionadas, auditoria, Loyalty, incidentes e registros de documentos.

Ele tambem popula os 41 modulos do mapeamento v47 e registra regras base de pricing, split, Ring-Fence e consentimento.

`database/postgres/005\_v47\_domain\_tables\_core\_first.sql` implanta tabelas dominio v47 adaptadas.

Esse arquivo cria Advisor, metas financeiras, teleterapia, uploads, chat, invoices, payrolls, Plug, afiliados e receipts sem recriar schemas legados.

`database/postgres/006\_v47\_column\_comments\_ptbr.sql` documenta as colunas das implantacoes v47.

Esse arquivo adiciona comentarios nativos para campos, triggers e integracoes criadas pelos scripts `004` e `005`.

`database/postgres/007\_v47\_module\_delivery\_automation.sql` implanta a camada de delivery automation dos 47 modulos.

Esse arquivo e gerado automaticamente por `scripts/valley\_module\_automation.py` a partir de `config/modules\_v47.json`.

Ele cria `module\_delivery\_registry`, `module\_evolution\_backlog` e `module\_automation\_runs` para controlar implantacao, desenvolvimento e evolucao.

`database/postgres/008\_v47\_foundation\_commerce\_operations.sql` implanta a camada operacional foundation de comercio e ERP.

Esse arquivo cria fornecedores, armazens, itens, lotes, movimentos append-only, listings, compras, linhas de compra, ordens de servico e contagens fisicas.

Ele cobre os primeiros contratos especificos dos modulos `REPLY`, `STOCK`, `WMS` e `MARKETPLACE`.

`database/mongodb/002\_v47\_log\_iot\_foundation.mongo.js` implanta a camada NoSQL foundation para tracking e sensores.

Esse arquivo cria validators e indices para `log\_tracking\_events`, `iot\_device\_registry`, `iot\_sensor\_events` e `warehouse\_sensor\_snapshots`.

Ele cobre os primeiros contratos específicos dos módulos `LOG`, `IOT` e snapshots operacionais do `WMS`.

`database/postgres/009\_v47\_tech\_legal\_platform\_contracts.sql` implanta a camada foundation de plataforma developer e jurídico.

Esse arquivo cria API clients, credenciais com hash, conectores, webhooks, tentativas append-only, contratos, partes, assinaturas append-only, disputas, eventos jurídicos append-only e fallback PIN por hash.

Ele cobre os primeiros contratos específicos dos módulos `TECH` e `LEGAL`.

`database/postgres/010\_v47\_rule\_growth\_marketplace\_runtime.sql` implanta o runtime comercial e de growth da arquitetura Valley.

Esse arquivo cria bindings e trilha do Rule Engine, storefronts, zonas de atendimento, controles de competitividade, snapshots de concorrência, contas/ledger de Pepitas, campanhas GOLD e validação de venda marketplace/física.

Ele cobre a evolução operacional dos módulos `MARKETPLACE`, `ADS` e do motor de cashback gamificado conectado ao ecossistema.

`database/postgres/011\_v47\_city\_ops\_delivery\_mobility\_security.sql` implanta a camada relacional de city ops e segurança.

Esse arquivo cria `delivery\_shipments`, `delivery\_shipment\_events`, `mobility\_trips`, `mobility\_trip\_events`, `security\_trusted\_contacts`, `security\_biometric\_credentials`, `security\_incidents` e `security\_incident\_events`.

Ele cobre a evolução operacional dos módulos `DELIVERY`, `MOBILITY` e `SECURITY`, mantendo biometria somente por hash e trilhas append-only de campo.

`database/postgres/012\_v47\_core\_services\_health\_jobs\_pharmacy\_events.sql` fecha o tier core de serviços transacionais.

Esse arquivo cria perfis e bookings de `SERVICES`, perfis e planos de `HEALTH`, vagas e engagements de `JOBS`, catálogo e dispensação de `PHARMACY` e programas/ingressos de `EVENTS`.

Ele também amplia `orders`, reforça ownership de wallet com trigger e cria trilhas append-only quando a prova operacional não pode ser mutada.

`database/postgres/013\_v47\_expansion\_assets\_civic\_impact.sql` abre a camada expansion do ecossistema.

Esse arquivo cria contratos para `DIGITAL`, `REAL\_ESTATE`, `EDU`, `VET`, `GOV`, `CHARITY` e `INSURANCE`.

Ele adiciona coleções e eventos de ativos, deals imobiliários, trilhas educacionais, casos veterinários, requests cívicos, fundo social e claims securitários com ledgers e trilhas append-only onde a prova exige imutabilidade.

`database/mongodb/003\_v47\_field\_ops\_security\_agenda.mongo.js` implanta a camada NoSQL de dispatch, frota, sinais de segurança e agenda.

Esse arquivo cria validators e índices para `delivery\_dispatch\_runs`, `fleet\_vehicle\_profiles`, `fleet\_maintenance\_events`, `security\_signal\_logs` e `agenda\_items`.

Ele cobre a evolução operacional dos módulos `DELIVERY`, `FLEET`, `SECURITY` e `AGENDA` sem empurrar payload de campo para o PostgreSQL.

`config/modules\_v47.json` é o registry canônico dos 47 módulos.

Esse arquivo centraliza código, número, domínio, tier, data home, dependências, integrações e descrição de cada módulo.

`modules/` contém uma pasta por módulo.

Cada pasta tem `README.md`, `STATUS.md` e `CONTRACT.md` gerados pela automação.

O `README.md` explica finalidade, dependências e trilha de implantação.

O `STATUS.md` guarda checklist evolutivo.

O `CONTRACT.md` define a fronteira operacional do módulo, política de dados, integrações e regras de evolução.

`output/module-roadmap/VALLEY\_MODULE\_ROADMAP.md` consolida a ordem de prioridade e o backlog macro dos 47 módulos.

`output/module-roadmap/VALLEY\_MODULE\_CONTRACTS.md` consolida a matriz técnica dos contratos operacionais dos 47 módulos.

`MANUAL\_ONLINE/DECSOES\_IMPLANTACAO\_V47.md` registra o que foi implantado, adaptado e descartado após leitura dos PDFs.

Esse arquivo evita ambiguidade futura e protege o projeto contra retorno acidental ao multi-schema legado.

`MANUAL\_ONLINE/01\_ARQUITETURA\_TECNICA\_MICROSSERVICOS\_EVENTOS.md` traduz o schema atual para boundaries de `microservices`, contratos síncronos e backbone de eventos.

`MANUAL\_ONLINE/02\_MODELAGEM\_BANCO\_DADOS.md` organiza a modelagem atual por domínio, aggregate root e ownership de dados.

`MANUAL\_ONLINE/03\_FLUXO\_COMPLETO\_APIS.md` descreve a superfície de APIs recomendada e os fluxos end-to-end mais importantes do produto.

`MANUAL\_ONLINE/04\_ROADMAP\_IMPLEMENTACAO\_SPRINTS.md` transforma o estado atual em backlog executivo e técnico por sprint.

`scripts/generate\_manual\_pdf.py` gera o PDF do manual a partir deste Markdown.

Esse script usa `reportlab`, consolida todos os arquivos `.md` de `MANUAL\_ONLINE` e deve ser executado novamente sempre que este manual mudar.

`scripts/valley\_module\_automation.py` automatiza o ciclo dos 47 módulos.

Esse script valida o registry, gera a documentação por módulo, gera contratos operacionais, gera roadmap e gera a migration SQL de delivery automation.

`database/migrations.json` define a ordem oficial de aplicação das migrations PostgreSQL e scripts MongoDB.

Esse manifesto evita execução fora de ordem e é consumido por `scripts/valley\_db\_orchestrator.py`.

`scripts/valley\_db\_orchestrator.py` valida ambiente, manifesto, SQL, MongoDB e registry.

Esse script também aplica migrations via `psql`, `mongosh` ou Docker Compose quando o banco estiver disponível.

`docker-compose.yml` define Postgres e Mongo locais para validação runtime.

`.env.example` documenta as variáveis locais `DATABASE\_URL`, `MONGODB\_URI` e `VALLEY\_AUTO\_APPLY`.

`database/README.md` explica a organização da pasta de banco e o fluxo de validação/aplicação.

`MANUAL\_ONLINE/OPERACAO\_AUTONOMA.md` documenta a esteira autônoma atual, política de descarte e fluxos Docker/local.

`output/deployment/VALLEY\_DEPLOYMENT\_STATUS.md` registra a última checagem operacional gerada pelo orquestrador.

Passo 1 - Nucleo de Identidade e Wallets

`users` é o no central absoluto do ecossistema.

Toda entidade operacional deve referenciar `users.user\_id` quando representar usuário, empresa, rider, admin ou sistema.

`pj\_profiles` especializa usuários do tipo `PJ`.

Essa tabela guarda dados empresariais, CNPJ, fiscal, responsável legal e KYB.

`rider\_profiles` especializa usuários do tipo `RIDER`.

Essa tabela guarda disponibilidade, veiculo, habilitacao, zona operacional e score.

`wallets` e o cofre financeiro por usuario.

Essa tabela separa BRL e NEX, com checks para impedir mistura indevida de saldos.

`led\_cards` liga identidade, wallet e cartao fisico/NFC.

Essa tabela protege unicidade de UID, serial NFC e tokenizacao.

Passo 2 - Ledger Financeiro, Smart Equity e Orders

`orders` e a tabela mestre para Food, Move e Dropship.

Ela guarda valores, status, participantes, enderecos, pagamento e colunas reservadas para cada dominio.

`transactions` e o ledger financeiro operacional.

Ele registra P2P, compras, pagamentos, repasses, refunds, chargebacks, fees, splits e escrow.

`transactions` e append-only.

Qualquer correcao deve ser feita por novo lancamento compensatorio, nunca por edicao ou exclusao.

`equity\_ledger` e o ledger de Smart Equity em token NEX.

Ele registra mint, allocate, transfer, lock, unlock, vest, burn, certificado e clausulas de drag/tag along.

`equity\_ledger` tambem e append-only.

O trigger de supply impede que o total liquido de `MINT - BURN` ultrapasse `1\_000\_000.00000000` NEX.

Passo 3 - MongoDB para IA, Social e Telemetria

`ai\_memory` guarda memoria contextual da Persona AI.

Ela integra cada memoria ao `user\_id` relacional e registra escopo, persona, modulo de origem e consentimento.

`social\_videos` guarda metadados do feed social.

Ela integra criador, dono operacional, links de comissao, produtos e contadores de engajamento.

`influencer\_metrics` guarda snapshots de campanha.

Ela integra influencer, campanha, periodo, funil, vendas brutas e comissao.

`telemetry\_logs` guarda eventos de alto volume.

Ela integra usuario, rider, device, evento, GeoJSON Point, sensor payload e correlation id.

Politica de Comentarios

Todo novo arquivo deve explicar em portugues simples o que cada bloco tecnico faz.

Termos tecnicos devem manter o ingles tecnico quando forem padroes de plataforma, como `validator`, `index`, `trigger`, `foreign key`, `append-only`, `JSON Schema`, `GeoJSON`, `ledger` e `wallet`.

Novos schemas devem comentar campos, constraints e integracoes principais.

Novos scripts devem comentar comandos, entradas, saidas e dependencias.

Quando a explicacao linha a linha tornar o arquivo ilegivel, a regra minima e comentar cada campo, cada comando e cada bloco funcional.

No PostgreSQL, os comentarios permanentes ficam em `COMMENT ON`, pois eles acompanham o schema dentro do banco e podem ser lidos por ferramentas de admin.

No MongoDB, os comentarios ficam no script `.mongo.js`, ao lado de validators, campos e indices.

Politica de Atualizacao Continua

Sempre que um arquivo de banco mudar, este manual deve ser atualizado no mesmo ciclo de trabalho.

Depois de atualizar o Markdown, o PDF deve ser regenerado em `output/pdf/VALLEY\_MANUAL\_ONLINE.pdf`.

O comando padrao para regenerar o PDF e:

```
python scripts/generate_manual_pdf.py
```

Se `reportlab` nao estiver instalado, instale a dependencia no ambiente Python antes de gerar o PDF.

Ordem Recomendada de SQL

A ordem recomendada de aplicacao em PostgreSQL limpo e:

```
001_core_identity_wallets.sql  
002_financial_ledger_equity_orders.sql
```



```
004_v47_control_plane_modules_rules.sql
005_v47_domain_tables_core_first.sql
003_database_comments_ptbr.sql
006_v47_column_comments_ptbr.sql
007_v47_module_delivery_automation.sql
008_v47_foundation_commerce_operations.sql
009_v47_tech_legal_platform_contracts.sql
010_v47_rule_growth_marketplace_runtime.sql
011_v47_city_ops_delivery_mobility_security.sql
012_v47_core_services_health_jobs_pharmacy_events.sql
013_v47_expansion_assets_civic_impact.sql
```

O script `003` documenta objetos dos passos iniciais.

O script `006` documenta objetos v47 e precisa ser executado depois de `004` e `005`.

O script `007` depende de `004` e `005`, porque usa as funcoes de trigger e integra o delivery registry ao nucleo ja criado.

O script `008` depende de `007`, porque usa `module\_delivery\_registry` como catalogo canonico dos 47 modulos.

O script `009` depende de `008`, porque continua a esteira foundation e reutiliza `module\_delivery\_registry`, `document\_records`, `orders`, `transactions` e `admin\_users`.

O script `010` depende de `009`, porque fecha o runtime comercial usando Rule Engine, campaigns, listings, wallets e trilhas append-only ja existentes.

O script `011` depende de `010`, porque reutiliza `module\_delivery\_registry`, `orders`, `document\_records`, `legal\_disputes` e o runtime operacional ja consolidado.

O script `012` depende de `011`, porque reforca ownership de wallet, expande `orders` e reutiliza a base operacional ja consolidada.

O script `013` depende de `012`, porque reutiliza `orders`, `document\_records`, `wallets`, `transactions` e contratos juridicos/operacionais do tier core.

O MongoDB deve aplicar `001\_ai\_social\_telemetry.mongo.js`, depois `002\_v47\_log\_iot\_foundation.mongo.js` e por fim `003\_v47\_field\_ops\_security\_agenda.mongo.js`.

Automacao Dos 47 Modulos

O ciclo padrao de automacao local e:

```
python scripts/valley_module_automation.py validate
python scripts/valley_module_automation.py sync
python scripts/valley_module_automation.py sql
python scripts/valley_db_orchestrator.py check
python scripts/valley_db_orchestrator.py report
python scripts/generate_manual_pdf.py
```

`validate` garante que existem exatamente 47 modulos numerados de 1 a 47.

`sync` cria ou atualiza `modules/<modulo>/README.md`, `modules/<modulo>/STATUS.md`, `modules/INDEX.md` e o roadmap.

`sync` tambem cria ou atualiza `modules/<modulo>/CONTRACT.md` e `output/module-roadmap/VALLEY\_MODULE\_CONTRACTS.md`.

`contracts` pode ser usado quando a alteracao desejada for apenas na camada de contratos operacionais.

`sql` gera `database/postgres/007\_v47\_module\_delivery\_automation.sql` a partir do registry.

`check` valida ambiente, manifesto, artefatos dos 47 modulos, SQL, MongoDB e registry sem exigir banco ativo.

`report` gera `output/deployment/VALLEY\_DEPLOYMENT\_STATUS.md` para registrar evidencias.

Depois disso, o PDF deve ser regenerado para manter a vertente executiva alinhada.

Contratos Operacionais

Cada `CONTRACT.md` define a fronteira inicial de um modulo.

Ele existe para impedir que o desenvolvimento crie tabelas duplicadas, schemas legados ou integracoes sem ponte com `users.user\_id`.

O contrato tambem registra quando usar PostgreSQL, MongoDB ou persistencia hibrida.

Os contratos nao substituem migration, teste ou regra de negocio.

Eles sao a camada de planejamento tecnico que vem antes de escrever schema especifico ou codigo de produto.

Esteira De Implantacao

`database/migrations.json` e a ordem oficial de banco.

O orquestrador usa esse manifesto para validar e aplicar scripts.

O ambiente atual permite validacao estatica completa e leitura automatica de `.env` ou `.env.example`, mas a aplicacao runtime ainda depende de `psql`, `mongosh` ou Docker daemon ativo.

Quando Docker estiver pronto, o fluxo local e:

```
python scripts/valley_db_orchestrator.py apply-compose
python scripts/valley_db_orchestrator.py report
python scripts/generate_manual_pdf.py
```

Quando houver banco externo e as variaveis estiverem no ambiente ou em `.env`, o fluxo minimo e:

```
python scripts/valley_db_orchestrator.py apply-postgres
python scripts/valley_db_orchestrator.py apply-mongo
```

Validacao Atual

Os scripts PostgreSQL foram revisados por validacao estatica do orquestrador.

Os artefatos `README.md`, `STATUS.md` e `CONTRACT.md` dos 47 modulos tambem foram validados pelo orquestrador.

O script MongoDB foi validado com `node --check`.

O registry dos 47 modulos foi validado por `scripts/valley\_module\_automation.py validate`.

O ultimo relatorio operacional fica em `output/deployment/VALLEY\_DEPLOYMENT\_STATUS.md`.

Nao houve execucao real em PostgreSQL porque `psql` nao esta instalado neste ambiente.

O script MongoDB foi escrito para `mongosh`, com `createCollection`, `collMod`, `JSON Schema Validation` e indices.

O PDF sera validado por leitura textual basica com `pypdf` depois da geracao.

Evolucao Foundation Mais Recente

Os modulos `REPLY`, `STOCK`, `LOG`, `WMS` e `MARKETPLACE` passaram para `implemented\_partial`.

`REPLY`, `STOCK`, `WMS` e `MARKETPLACE` receberam schema PostgreSQL especifico em `008\_v47\_foundation\_commerce\_operations.sql`.

`LOG` recebeu schema MongoDB especifico em `002\_v47\_log\_iot\_foundation.mongo.js`.

`IOT` foi expandido na mesma migration MongoDB com registry de devices e eventos de sensores.

Evolucao TECH e LEGAL Mais Recente

Os modulos `TECH` e `LEGAL` passaram para `implemented\_partial`.

`TECH` recebeu schema PostgreSQL específico para API clients, credenciais por hash, conectores, webhooks e uso diário de API.

`LEGAL` recebeu schema PostgreSQL específico para contratos, partes, assinaturas, disputas, eventos jurídicos e fallback PIN por hash.

Credenciais, webhook secrets, IPs, device fingerprints e fallback PIN não são armazenados em texto bruto.

Evolução Rule Engine, Pepitas e Ads Mais Recente

Os módulos `MARKETPLACE` e `ADS` avançaram na camada operacional ligada a growth e validação de venda.

`MARKETPLACE` recebeu storefronts, zonas de atendimento, controles de competitividade e snapshots append-only de concorrência.

`ADS` recebeu campanhas `GOLD`, eventos append-only e validação de venda com geolocalização, pedido ou POS.

O Rule Engine ganhou bindings runtime e trilha append-only de execução, sem duplicar `business\_rule\_definitions` e `business\_rule\_versions`.

As `Pepitas` passaram a ter conta consolidada, cap de até 50 por cento do lucro líquido por venda validada e ledger append-only.

Evolução City Ops, Fleet, Security e Agenda Mais Recente

Os módulos `DELIVERY`, `FLEET`, `SECURITY` e `AGENDA` passaram para `implemented\_partial`.

`DELIVERY` recebeu camada relacional de shipments e trilha append-only de eventos, além de camada NoSQL para dispatch e matching de riders.

`MOBILITY` foi aprofundado com trips relacionais, checkpoints append-only e coerência forte com `orders`, `wallets` e `rider\_profiles`.

`SECURITY` recebeu contatos confiáveis, incidentes, eventos append-only, sinais de alto volume e biometria persistida somente por hash.

`AGENDA` recebeu collection própria para lembretes, tarefas, follow-ups e sincronização com `ADVISOR` e `CHAT`.

Evolução Creator, Influencers, Advisor e Conteúdo

`INFLUENCERS` ja pode operar sobre a pipeline compartilhada de `creator\_uploads`, `social\_videos`, `influencer\_metrics` e `affiliate\_referrals`, sem precisar de tabela exclusiva com nome do modulo.

`ADVISOR` ja tem camada hibrida real com `advisor\_insights`, `financial\_goals`, `teletherapy\_sessions` e `ai\_memory`, alem de integracao natural com `AGENDA` e `CHAT`.

`NEWS\_PODCAST` continua `PLANNED` no registry, mas o MVP ja pode nascer sobre `creator\_uploads`, `social\_videos` e `document\_records`, sem criar schema editorial paralelo antes da prova de produto.

\n\n\n# Fonte: MANUAL_ONLINE\01_ARQUITETURA_TECNICA_MICROSSERVICOS_EVENTOS.md\n\n\n# Arquitetura Tecnica - Microservices e Eventos

Este documento transforma o banco atual do Valley em uma arquitetura tecnica de servicos, contratos sincronos e eventos assincronos.

O objetivo aqui nao e inventar outro produto.

O objetivo e traduzir o que ja existe em `database/postgres/001-011` e `database/mongodb/001-003` para uma topologia de runtime coerente.

Principios

- `public.users.user\_id` continua sendo o no central absoluto.
- `wallets`, `transactions`, `equity\_ledger`, `pepita\_ledger` e trilhas append-only continuam como source of truth relacional.
- MongoDB continua reservado para telemetria, social, dispatch, sinais de seguranca, agenda e payload semi-estruturado.
- Microservices compartilham o mesmo cluster PostgreSQL na fase atual, mas cada servico tem ownership logico do seu conjunto de tabelas.
- Integracao entre servicos deve acontecer por API ou evento; escrita direta em tabela de outro dominio e anti-pattern.
- Segredo bruto, biometria bruta, API key bruta, webhook secret bruto e fallback PIN bruto continuam descartados.

Topologia Recomendada

Camada de entrada:

- `web-admin-bff` para painel administrativo, operacao, regras e governanca.
- `consumer-api-gateway` para app do usuario, merchant app, rider app e parceiros.
- `partner-api-gateway` para clients externos do Valley Tech com `API key hash`, `JWT`, `OAuth2` ou `mTLS`.

Camada de servicos:

- `identity-access-service`
- `wallet-ledger-service`
- `catalog-commerce-service`
- `orders-orchestration-service`
- `rules-growth-service`
- `delivery-dispatch-service`
- `mobility-service`
- `security-response-service`
- `tech-platform-service`
- `legal-docs-service`
- `social-media-service`
- `ai-agenda-service`
- `admin-control-plane-service`

Camada de dados:

- `postgres-core` como source of truth transacional.
- `mongo-ops` para alto volume, sinais, dispatch, social e agenda.
- `object-storage` para recibos, provas, documentos e anexos, sempre referenciados por URL/checksum no banco.

Camada de mensageria:

- `event-bus` recomendado em `NATS JetStream` na fase inicial por simplicidade operacional, reprocessamento e baixo overhead.
- Padrao de entrega: `at-least-once`.
- Padrao de protecao: `outbox` no PostgreSQL para publicacao confiavel de eventos de dominio.
- Padrao de consumo: `idempotent consumer` com `event\_id`, `correlation\_id` e chave de deduplicacao.

Ownership por Servico

1. identity-access-service

Responsabilidade:

- onboarding, KYC/KYB, perfis PF/PJ/Rider, identidade forte e cartoes LED.

Ownership relacional:

- `users`

- `pj\_profiles`
- `rider\_profiles`
- `led\_cards`
- `security\_trusted\_contacts`
- `security\_biometric\_credentials`

Eventos emitidos:

- `identity.user.created`
- `identity.user.kyc\_updated`
- `identity.rider.activated`
- `identity.led\_card.assigned`

2. wallet-ledger-service

Responsabilidade:

- carteiras, autorizacao, settle, refund, split, escrow, equity e saldo gamificado financeiro.

Ownership relacional:

- `wallets`
- `transactions`
- `equity\_ledger`
- `plug\_transactions`

Eventos emitidos:

- `wallet.created`
- `ledger.transaction.authorized`
- `ledger.transaction.settled`
- `ledger.transaction.failed`
- `equity.entry.created`

3. catalog-commerce-service

Responsabilidade:

- merchant onboarding operacional, estoque, preco, dropshipping, listings, storefronts e zonas.

Ownership relacional:

- `suppliers`
- `warehouses`
- `inventory\_items`
- `inventory\_lots`
- `inventory\_movements`
- `marketplace\_listings`
- `merchant\_storefronts`
- `merchant\_service\_zones`
- `marketplace\_listing\_controls`
- `marketplace\_competitor\_snapshots`

Eventos emitidos:

- `catalog.item.created`
- `catalog.listing.created`
- `catalog.listing.price\_checked`
- `catalog.listing.activated`
- `catalog.listing.auto\_paused`
- `catalog.stock.changed`

4. orders-orchestration-service

Responsabilidade:

- pedido mestre, procurement, service work order, faturamento de fluxo e orquestracao da jornada de compra.

Ownership relacional:

- `orders`
- `procurement\_orders`
- `procurement\_order\_items`
- `service\_work\_orders`
- `business\_invoices`
- `business\_payrolls`
- `affiliate\_referrals`

Eventos emitidos:

- `order.placed`

- `order.confirmed`
- `order.cancelled`
- `order.refunded`
- `order.dispatched`

5. rules-growth-service

Responsabilidade:

- Rule Engine, campanhas, Pepitas, GOLD, validacao de venda e runtime de incentivos.

Ownership relacional:

- `business\_rule\_definitions`
- `business\_rule\_versions`
- `business\_rule\_audit`
- `rule\_runtime\_bindings`
- `rule\_execution\_events`
- `gamification\_campaigns`
- `points\_ledger`
- `pepita\_accounts`
- `pepita\_ledger`
- `gold\_campaigns`
- `gold\_campaign\_events`
- `sale\_validation\_events`

Eventos emitidos:

- `rules.binding.activated`
- `rules.execution.recorded`
- `growth.gold\_campaign.created`
- `growth.sale.validated`
- `loyalty.pepita.granted`
- `loyalty.pepita.redeemed`

6. delivery-dispatch-service

Responsabilidade:

- shipment, dispatch, alocação de rider, checkpoint, ETA e prova de entrega.

Ownership relacional:

- `delivery\_shipments`
- `delivery\_shipment\_events`

Ownership MongoDB:

- `delivery\_dispatch\_runs`
- `log\_tracking\_events`

Eventos emitidos:

- `delivery.shipment.created`
- `delivery.dispatch.started`
- `delivery.rider.assigned`
- `delivery.shipment.checkpoint`
- `delivery.shipment.delivered`

7. mobility-service

Responsabilidade:

- corrida urbana, checkpoint de viagem, tarificacao operacional e integracao rider/passageiro.

Ownership relacional:

- `mobility\_trips`
- `mobility\_trip\_events`

Ownership MongoDB:

- `telemetry\_logs`
- `fleet\_vehicle\_profiles`
- `fleet\_maintenance\_events`

Eventos emitidos:

- `mobility.trip.created`
- `mobility.trip.matched`
- `mobility.trip.started`
- `mobility.trip.completed`

- `mobility.trip.incident\_opened`

8. security-response-service

Responsabilidade:

- SOS, sinais de risco, incidente, escalonamento e evidencias de resposta.

Ownership relacional:

- `security\_incidents`
- `security\_incident\_events`

Ownership MongoDB:

- `security\_signal\_logs`

Eventos emitidos:

- `security.signal.opened`
- `security.incident.created`
- `security.incident.escalated`
- `security.incident.resolved`

9. tech-platform-service

Responsabilidade:

- API clients, credenciais por hash, conectores, webhooks e uso diario de API.

Ownership relacional:

- `tech\_api\_clients`
- `tech\_api\_credentials`
- `tech\_integration\_connectors`
- `tech\_webhook\_subscriptions`
- `tech\_webhook\_delivery\_attempts`
- `tech\_api\_usage\_daily`

Eventos emitidos:

- `tech.client.created`

- `tech.webhook.subscribed`
- `tech.webhook.delivery\_recorded`

10. legal-docs-service

Responsabilidade:

- contratos, assinaturas, disputa, fallback PIN por hash, recibos e documentos rastreaveis.

Ownership relacional:

- `legal\_contracts`
- `legal\_contract\_parties`
- `legal\_signatures`
- `legal\_disputes`
- `legal\_audit\_events`
- `legal\_fallback\_pin\_credentials`
- `document\_records`
- `docs\_receipts`

Eventos emitidos:

- `legal.contract.created`
- `legal.contract.signed`
- `legal.dispute.opened`
- `docs.document.generated`
- `docs.receipt.generated`

11. social-media-service

Responsabilidade:

- feed, creator uploads, influencer metrics, midia e atribuicao social.

Ownership relacional:

- `creator\_uploads`

Ownership MongoDB:

- `social\_videos`

- `influencer\_metrics`

Eventos emitidos:

- `media.upload.created`
- `social.video.published`
- `social.campaign.metric\_aggregated`

12. ai-agenda-service

Responsabilidade:

- memoria contextual, advisor, chat, follow-up e agenda inteligente.

Ownership relacional:

- `advisor\_insights`
- `financial\_goals`
- `teletherapy\_sessions`
- `chat\_conversations`
- `chat\_messages`

Ownership MongoDB:

- `ai\_memory`
- `agenda\_items`

Eventos emitidos:

- `ai.memory.created`
- `advisor.insight.generated`
- `chat.message.created`
- `agenda.item.scheduled`

13. admin-control-plane-service

Responsabilidade:

- governanca, RBAC/ABAC, modulo registry, backlog, automacao e observabilidade de negocio.

Ownership relacional:

- `module\_catalog`
- `admin\_users`
- `admin\_permissions`
- `observability\_incidents`
- `admin\_action\_audit`
- `module\_delivery\_registry`
- `module\_evolution\_backlog`
- `module\_automation\_runs`

Eventos emitidos:

- `admin.action.recorded`
- `module.delivery.updated`
- `observability.incident.created`

Contratos Sincronos

Padrao recomendado:

- REST JSON para borda externa.
- gRPC opcional apenas entre servicos de alta frequencia quando houver ganho real.
- `Idempotency-Key` obrigatoria em create financeiro, funding GOLD, criacao de order, dispatch e documento.
- `X-Correlation-Id` obrigatoria em requests distribuidados.

Chamadas sincronas que fazem sentido:

- `consumer-api-gateway -> identity-access-service`
- `consumer-api-gateway -> wallet-ledger-service`
- `web-admin-bff -> admin-control-plane-service`
- `web-admin-bff -> rules-growth-service`
- `partner-api-gateway -> tech-platform-service`
- `orders-orchestration-service -> wallet-ledger-service` para authorize/settle
- `catalog-commerce-service -> rules-growth-service` para price-check e publish gate
- `delivery-dispatch-service -> security-response-service` para escalonamento imediato

Backbone de Eventos

Topicos de dominio recomendados:

- `identity.user.created`
- `identity.user.kyc\_updated`
- `wallet.transaction.settled`
- `catalog.listing.price\_checked`
- `catalog.listing.activated`
- `order.placed`
- `order.confirmed`
- `growth.gold\_campaign.created`
- `growth.sale.validated`
- `loyalty.pépita.granted`
- `delivery.shipment.status\_changed`
- `mobility.trip.status\_changed`
- `security.signal.opened`
- `legal.contract.signed`
- `docs.document.generated`
- `agenda.item.scheduled`

Formato minimo do envelope:

- `event\_id`
- `event\_name`
- `event\_version`
- `occurred\_at`
- `producer\_service`
- `correlation\_id`
- `causation\_id`
- `tenant\_scope` quando existir
- `payload`

Padroes de Consistencia

- `Postgres -> Outbox -> Event Bus` para eventos de dominio.
- `MongoDB` nao publica evento diretamente sem passar por servico dono.
- `append-only` continua obrigatorio para ledger, auditoria, checkpoints, assinaturas, snapshots de concorrencia e eventos de seguranga.
- Processos compensatorios substituem `UPDATE/DELETE` em livros criticos.

O que foi descartado

- `database-per-service` agora: descartado na fase atual porque a base existente já está consolidada em `public` e separar fisicamente agora criaria retrabalho.
- `event sourcing global`: descartado porque o projeto já tem source of truth relacional clara em tabelas mutáveis + append-only.
- `shared writes` entre serviços: descartado; o serviço consumidor deve pedir por API ou reagir por evento.
- `telemetria no Postgres`: descartado; alto volume continua no MongoDB.
- `segredo e biometria bruta em evento`: descartado por risco técnico e jurídico.

Estado alvo por fase

Fase 1:

- cluster PostgreSQL único
- cluster Mongo único
- microservices com ownership lógico
- `API Gateway` único
- `event-bus` único

Fase 2:

- separar workloads de leitura
- materialized views e read models por domínio
- rate limit por módulo e cliente externo
- tracing distribuído fim a fim

Fase 3:

- split físico dos domínios mais quentes
- `data products` para analytics
- automação de replay de eventos e recovery operacional

\n\n\n# Fonte: MANUAL_ONLINE\02_MODELAGEM_BANCO_DADOS.md\n\n# Modelagem de Banco de Dados

Este documento organiza a modelagem atual do Valley por domínio funcional.

Ele não substitui as migrations.

Ele existe para explicar como as tabelas e coleções já criadas se encaixam como modelo de negócio e modelo operacional.

Regras Mestras

- Tudo que representa identidade, dinheiro, contrato, order, disputa, documento ou trilha juridica fica no PostgreSQL.
- Tudo que representa alto volume, feed, telemetria, dispatch, sinais e agenda fica no MongoDB.
- `user\_id` e a ancora universal entre os dois bancos.
- `UUID` e o identificador padrao.
- `TIMESTAMPZ` ou `date` padronizam o tempo operacional.
- BRL usa `DECIMAL(18,4)`.
- `\$NEX` usa `DECIMAL(18,8)`.

Cobertura Atual por Migration

PostgreSQL:

- `001` identidade e wallets
- `002` orders, transactions e equity
- `004` control plane, regras, campanhas e docs base
- `005` tabelas de dominio complementares
- `007` registry de 47 modulos
- `008` comercio, estoque e marketplace
- `009` tech e legal
- `010` rule runtime, Pepitas, GOLD e growth
- `011` delivery, mobility e security

MongoDB:

- `mongo-001` IA, social, influencer e telemetria
- `mongo-002` log, IoT e snapshots WMS
- `mongo-003` dispatch, frota, sinais de seguranca e agenda

Nucleo Absoluto

Identidade

Tabelas:

- `users`
- `pj\_profiles`
- `rider\_profiles`
- `led\_cards`

Papel:

- `users` define a identidade unica do ecossistema.
- `pj\_profiles` especializa merchant, empresa e ator institucional.
- `rider\_profiles` especializa operador de campo.
- `led\_cards` liga identidade forte, NFC e wallet.

Dependencias fortes:

- todo dominio relacional aponta para `users.user\_id`
- `pj\_profiles` e `rider\_profiles` dependem do `user\_kind`

Financeiro

Tabelas:

- `wallets`
- `transactions`
- `equity\_ledger`
- `plug\_transactions`

Papel:

- `wallets` e saldo mutavel e reconciliavel.
- `transactions` e ledger append-only de dinheiro.
- `equity\_ledger` e ledger append-only societario.
- `plug\_transactions` especializa adquirencia/maquininha.

Control Plane e Governanca

Tabelas:

- `module\_catalog`
- `module\_delivery\_registry`
- `module\_evolution\_backlog`

- `module\_automation\_runs`
- `admin\_users`
- `admin\_permissions`
- `admin\_action\_audit`
- `observability\_incidents`

Papel:

- governanca do backlog dos 47 modulos
- RBAC/ABAC
- auditoria do painel admin
- evidencia operacional da automacao

Rule Engine e Growth

Tabelas:

- `business\_rule\_definitions`
- `business\_rule\_versions`
- `business\_rule\_audit`
- `rule\_runtime\_bindings`
- `rule\_execution\_events`
- `gamification\_campaigns`
- `points\_ledger`
- `pepita\_accounts`
- `pepita\_ledger`
- `gold\_campaigns`
- `gold\_campaign\_events`
- `sale\_validation\_events`

Papel:

- separar definicao da regra de sua execucao
- manter auditoria de governance
- permitir runtime por modulo, merchant, listing, order ou campanha
- controlar Pepitas e GOLD com cap financeiro e venda validada

Relacoes importantes:

- `business\_rule\_definitions` -> `business\_rule\_versions` -> `rule\_runtime\_bindings` -> `rule\_execution\_events`
- `gold\_campaigns` -> `gold\_campaign\_events`
- `sale\_validation\_events` -> `gold\_campaign\_events` -> `pepita\_ledger`
- `pepita\_accounts` -> `pepita\_ledger`

Regra-chave:

- `pepita\_cap\_brl` em `sale\_validation\_events` limita o incentivo a ate 50 por cento do lucro liquido de referencia.

Comercio e Marketplace

Tabelas:

- `suppliers`
- `warehouses`
- `inventory\_items`
- `inventory\_lots`
- `inventory\_movements`
- `marketplace\_listings`
- `merchant\_storefronts`
- `merchant\_service\_zones`
- `marketplace\_listing\_controls`
- `marketplace\_competitor\_snapshots`

Papel:

- `inventory\_\*` e o backbone do item, lote e saldo operacional
- `marketplace\_listings` expande item para anuncio comercial
- `merchant\_storefronts` e `merchant\_service\_zones` modelam a operacao geolocalizada
- `marketplace\_listing\_controls` e `marketplace\_competitor\_snapshots` modelam a regra "so publica se for competitivo e rentavel"

Append-only neste bloco:

- `inventory\_movements`
- `marketplace\_competitor\_snapshots`

Orders e Backoffice Comercial

Tabelas:

- `orders`
- `procurement\_orders`
- `procurement\_order\_items`
- `service\_work\_orders`
- `business\_invoices`
- `business\_payrolls`
- `affiliate\_referrals`
- `docs\_receipts`
- `document\_records`

Papel:

- `orders` e a entidade mae para Food, Move e Dropship
- procurement e service work orders cobrem backoffice e campo
- invoices, payroll e receipts ligam business, docs e financeiro

Tech e Legal

Tabelas:

- `tech\_api\_clients`
- `tech\_api\_credentials`
- `tech\_integration\_connectors`
- `tech\_webhook\_subscriptions`
- `tech\_webhook\_delivery\_attempts`
- `tech\_api\_usage\_daily`
- `legal\_contracts`
- `legal\_contract\_parties`
- `legal\_signatures`
- `legal\_disputes`
- `legal\_audit\_events`
- `legal\_fallback\_pin\_credentials`

Papel:

- Tech modela clientes externos, credenciais seguras e webhooks
- Legal modela contratos, assinatura, mediação e disputa

- Docs complementa Legal com `document\_records` e `docs\_receipts`

Append-only neste bloco:

- `tech\_webhook\_delivery\_attempts`
- `legal\_signatures`
- `legal\_audit\_events`

Delivery, Mobility e Security

Tabelas:

- `delivery\_shipments`
- `delivery\_shipment\_events`
- `mobility\_trips`
- `mobility\_trip\_events`
- `security\_trusted\_contacts`
- `security\_biometric\_credentials`
- `security\_incidents`
- `security\_incident\_events`

Papel:

- `delivery\_shipments` amarra order, rider, wallet e prova de entrega
- `mobility\_trips` amarra corrida, rider, passageiro e tarifa
- `security\_\*` cobre biometria por hash, contatos de emergencia e incidentes

Append-only neste bloco:

- `delivery\_shipment\_events`
- `mobility\_trip\_events`
- `security\_incident\_events`

MongoDB - IA, Social e Operacao de Campo

IA e agenda

Colecoes:

- `ai\_memory`

- `agenda\_items`

Papel:

- memoria contextual da persona
- follow-up, lembrete, tarefa e evento de agenda

Social e creator economy

Colecoes:

- `social\_videos`
- `influencer\_metrics`

Papel:

- feed social
- performance de campanha
- atribuicao e monetizacao de creator

Telemetria e IoT

Colecoes:

- `telemetry\_logs`
- `iot\_device\_registry`
- `iot\_sensor\_events`
- `warehouse\_sensor\_snapshots`
- `log\_tracking\_events`

Papel:

- GPS, sensor, tracking, temperatura, bateria, velocidade, correlograma de campo

Dispatch, frota e seguranca

Colecoes:

- `delivery\_dispatch\_runs`
- `fleet\_vehicle\_profiles`
- `fleet\_maintenance\_events`

- `security\_signal\_logs`

Papel:

- matching de rider
- cadastro vivo de veiculo
- manutencao preventiva
- sinais de alto volume para resposta rapida

Integracao Postgres + Mongo

Padrao de ponte:

- `UUID` tecnico em string no Mongo
- `correlation\_id` para rastrear request, evento, log e incidente
- `document\_id`, `order\_id`, `transaction\_id`, `trip\_id`, `shipment\_id` e `incident\_id` como referencias cruzadas logicas

Regra pratica:

- o estado juridico e financeiro fica no Postgres
- o sinal volumoso e a inteligencia operacional ficam no Mongo

Entidades que merecem Aggregate Root

Aggregate roots mais claros hoje:

- `users`
- `wallets`
- `orders`
- `marketplace\_listings`
- `merchant\_storefronts`
- `gold\_campaigns`
- `pepita\_accounts`
- `delivery\_shipments`
- `mobility\_trips`
- `security\_incidents`
- `legal\_contracts`
- `tech\_api\_clients`

O que nao entra agora

- multi-schema legado
- replica logica de `platform.users`
- event store unico para tudo
- tabela de telemetria em Postgres
- segredo bruto em qualquer banco

Diretriz para proximas tabelas e colecoes

Se um novo modulo precisar nascer:

- primeiro escolhe owner service
- depois escolhe data home
- depois escolhe root entity
- depois define eventos de entrada e saida
- so entao cria migration ou collection

\n\n\n# Fonte: MANUAL_ONLINE\03_FLUXO_COMPLETO_APIS.md\n\n# Fluxo Completo de APIs

Este documento descreve a superficie de API recomendada para o estado atual do Valley.

Ele nao e um `OpenAPI` final.

Ele e o mapa funcional de endpoints, comandos principais e eventos emitidos para cada fluxo de negocio.

Convencoes

Prefixo:

- `/v1`

Headers obrigatorios em fluxos criticos:

- `Authorization`
- `X-Correlation-Id`
- `Idempotency-Key` em create financeiro, order, GOLD, dispatch e documento

Autenticacao:

- usuario final: `JWT`
- admin: `JWT` com escopo admin + RBAC/ABAC
- parceiro externo: `API key hash`, `JWT`, `OAuth2` ou `mTLS`

Resposta padrao:

- `request\_id`
- `correlation\_id`
- `data`
- `errors`

Grupos de API

Identity API

- `POST /v1/id/users`
- `GET /v1/id/users/{user\_id}`
- `PATCH /v1/id/users/{user\_id}`
- `POST /v1/id/users/{user\_id}/kyc/submissions`
- `POST /v1/id/users/{user\_id}/pj-profile`
- `POST /v1/id/users/{user\_id}/rider-profile`
- `POST /v1/id/users/{user\_id}/led-cards`

Pay API

- `POST /v1/pay/wallets/bootstrap`
- `GET /v1/pay/wallets/{wallet\_id}`
- `POST /v1/pay/transactions/authorize`
- `POST /v1/pay/transactions/settle`
- `POST /v1/pay/transactions/refund`
- `POST /v1/pay/transactions/adjust`
- `GET /v1/pay/transactions/{transaction\_id}`

Marketplace API

- `POST /v1/marketplace/storefronts`
- `POST /v1/marketplace/storefronts/{storefront\_id}/zones`
- `POST /v1/marketplace/items`

- `POST /v1/marketplace/listings`
- `POST /v1/marketplace/listings/{listing\_id}/price-check`
- `POST /v1/marketplace/listings/{listing\_id}/publish`
- `GET /v1/marketplace/listings`
- `GET /v1/marketplace/listings/{listing\_id}`

Orders API

- `POST /v1/orders`
- `POST /v1/orders/checkout`
- `GET /v1/orders/{order\_id}`
- `POST /v1/orders/{order\_id}/confirm`
- `POST /v1/orders/{order\_id}/cancel`
- `POST /v1/orders/{order\_id}/refund`

Growth API

- `POST /v1/rules`
- `POST /v1/rules/{rule\_id}/versions`
- `POST /v1/rules/{rule\_id}/bindings`
- `POST /v1/rules/evaluate`
- `POST /v1/growth/gold-campaigns`
- `POST /v1/growth/sale-validations`
- `POST /v1/growth/gold-campaign-events`
- `POST /v1/loyalty/pepita-ledger`
- `GET /v1/loyalty/accounts/{user\_id}`

Delivery API

- `POST /v1/delivery/shipments`
- `GET /v1/delivery/shipments/{shipment\_id}`
- `POST /v1/delivery/dispatch`
- `POST /v1/delivery/shipments/{shipment\_id}/events`

Mobility API

- `POST /v1/mobility/trips`
- `GET /v1/mobility/trips/{trip\_id}`

- `POST /v1/mobility/trips/{trip\_id}/events`

Security API

- `POST /v1/security/signals/sos`

- `POST /v1/security/incidents`

- `POST /v1/security/incidents/{incident\_id}/events`

- `POST /v1/security/users/{user\_id}/trusted-contacts`

- `POST /v1/security/users/{user\_id}/biometric-credentials`

Tech API

- `POST /v1/tech/api-clients`

- `POST /v1/tech/api-clients/{api\_client\_id}/credentials`

- `POST /v1/tech/connectors`

- `POST /v1/tech/webhooks/subscriptions`

- `POST /v1/tech/webhooks/{subscription\_id}/deliveries`

Legal e Docs API

- `POST /v1/legal/contracts`

- `POST /v1/legal/contracts/{contract\_id}/parties`

- `POST /v1/legal/contracts/{contract\_id}/signatures`

- `POST /v1/legal/disputes`

- `POST /v1/docs/records`

- `POST /v1/docs/receipts`

AI e Agenda API

- `POST /v1/ai/memory`

- `GET /v1/ai/memory/{user\_id}`

- `POST /v1/chat/conversations`

- `POST /v1/chat/conversations/{conversation\_id}/messages`

- `POST /v1/agenda/items`

- `PATCH /v1/agenda/items/{agenda\_item\_id}`

Fluxo 1 - Onboarding, KYC e Wallet Bootstrap

Passo 1:

- `POST /v1/id/users`
- cria `users`
- opcionalmente cria `pj\_profiles` ou `rider\_profiles`

Passo 2:

- `POST /v1/id/users/{user\_id}/kyc/submissions`
- atualiza `kyc\_status`
- emite `identity.user.kyc\_updated`

Passo 3:

- `POST /v1/pay/wallets/bootstrap`
- cria carteiras base do usuario
- emite `wallet.created`

Resposta funcional esperada:

- `user\_id`
- `kyc\_status`
- wallets iniciais
- `led\_card\_default\_id` se existir

Fluxo 2 - Merchant, Storefront, Listing e Price Check

Passo 1:

- `POST /v1/marketplace/storefronts`
- cria `merchant\_storefronts`

Passo 2:

- `POST /v1/marketplace/storefronts/{storefront\_id}/zones`
- cria `merchant\_service\_zones`

Passo 3:

- `POST /v1/marketplace/items`
- cria `inventory\_items`

Passo 4:

- `POST /v1/marketplace/listings`
- cria `marketplace\_listings`
- cria `marketplace\_listing\_controls` em estado inicial

Passo 5:

- `POST /v1/marketplace/listings/{listing\_id}/price-check`
- grava `marketplace\_competitor\_snapshots`
- atualiza `marketplace\_listing\_controls`
- opcionalmente chama `POST /v1/rules/evaluate`

Eventos emitidos:

- `catalog.listing.created`
- `catalog.listing.price\_checked`
- `catalog.listing.activated` ou `catalog.listing.auto\_paused`

Fluxo 3 - Checkout, Pagamento e Geracao de Pedido

Passo 1:

- `POST /v1/orders/checkout`
- cria `orders` em `PLACED`

Passo 2:

- `POST /v1/pay/transactions/authorize`
- cria `transactions` em `AUTHORIZED` ou `PENDING`

Passo 3:

- `POST /v1/pay/transactions/settle`
- liquida o pagamento
- atualiza `orders.payment\_transaction\_id`

Passo 4:

- `POST /v1/orders/{order\_id}/confirm`
- transiciona pedido para `CONFIRMED`

Passo 5:

- `POST /v1/docs/receipts`
- grava `docs\_receipts`
- opcionalmente gera `document\_records`

Eventos emitidos:

- `order.placed`
- `ledger.transaction.authorized`
- `ledger.transaction.settled`
- `order.confirmed`
- `docs.receipt.generated`

Fluxo 4 - GOLD, Validacao de Venda e Pepita

Passo 1:

- `POST /v1/growth/gold-campaigns`
- cria `gold\_campaigns`
- opcionalmente referencia `gamification\_campaigns`

Passo 2:

- `POST /v1/pay/transactions/authorize`
- reserva funding da campanha

Passo 3:

- venda ocorre por marketplace ou venda fisica validavel

Passo 4:

- `POST /v1/growth/sale-validations`
- cria `sale\_validation\_events`
- valida `order\_id`, `transaction\_id`, `GPS` ou `sale\_reference\_code`
- calcula `pepita\_cap\_brl`

Passo 5:

- `POST /v1/growth/gold-campaign-events`

- cria `gold\_campaign\_events`
- separa `valley\_revenue\_amount\_brl` e `pepita\_amount\_brl`

Passo 6:

- `POST /v1/loyalty/pepita-ledger`
- cria `pepita\_ledger`
- atualiza `pepita\_accounts`

Eventos emitidos:

- `growth.gold\_campaign.created`
- `growth.sale.validated`
- `loyalty.pepita.granted`

Regra de bloqueio:

- sem venda validada, nao existe liquidacao GOLD
- sem margem liquida de referencia, nao existe Pepita

Fluxo 5 - Delivery e Dispatch

Passo 1:

- `POST /v1/delivery/shipments`
- cria `delivery\_shipments`

Passo 2:

- `POST /v1/delivery/dispatch`
- cria ou atualiza `delivery\_dispatch\_runs` no MongoDB
- executa matching de riders

Passo 3:

- `POST /v1/delivery/shipments/{shipment\_id}/events`
- grava `delivery\_shipment\_events`
- atualiza `delivery\_shipments.shipment\_status`

Passo 4:

- prova de entrega pode gerar `document\_records`

Eventos emitidos:

- `delivery.shipment.created`
- `delivery.dispatch.started`
- `delivery.rider.assigned`
- `delivery.shipment.delivered`

Fluxo 6 - Mobility e SOS

Passo 1:

- `POST /v1/mobility/trips`
- cria `mobility\_trips`

Passo 2:

- `POST /v1/mobility/trips/{trip\_id}/events`
- grava `mobility\_trip\_events`
- status avança no lifecycle

Passo 3:

- se houver risco, `POST /v1/security/signals/sos`
- grava `security\_signal\_logs`

Passo 4:

- `POST /v1/security/incidents`
- cria `security\_incidents`

Passo 5:

- `POST /v1/security/incidents/{incident\_id}/events`
- grava `security\_incident\_events`

Eventos emitidos:

- `mobility.trip.started`
- `security.signal.opened`
- `security.incident.created`

Fluxo 7 - Contrato, Assinatura e Documento

Passo 1:

- `POST /v1/legal/contracts`
- cria `legal\_contracts`

Passo 2:

- `POST /v1/legal/contracts/{contract\_id}/parties`
- cria `legal\_contract\_parties`

Passo 3:

- `POST /v1/legal/contracts/{contract\_id}/signatures`
- cria `legal\_signatures`
- anexa `document\_records` quando houver

Passo 4:

- se houver conflito, `POST /v1/legal/disputes`
- cria `legal\_disputes`

Eventos emitidos:

- `legal.contract.created`
- `legal.contract.signed`
- `legal.dispute.opened`

Fluxo 8 - Admin de Regras e Runtime

Passo 1:

- `POST /v1/rules`
- cria `business\_rule\_definitions`

Passo 2:

- `POST /v1/rules/{rule\_id}/versions`
- cria `business\_rule\_versions`

Passo 3:

- `POST /v1/rules/{rule\_id}/bindings`
- cria `rule\_runtime\_bindings`

Passo 4:

- chamadas de negocio usam `POST /v1/rules/evaluate`
- gravam `rule\_execution\_events`

Eventos emitidos:

- `rules.binding.activated`
- `rules.execution.recorded`

Sequencia Minima por Aplicativo

App do usuario:

- onboarding
- browse de listing
- checkout
- saldo e Pepitas
- agenda e chat

App do merchant:

- storefront
- item/listing
- price-check
- GOLD
- venda fisica validada
- receipts/docs

App do rider:

- dispatch
- shipment/trip event
- incident/SOS

Painel admin:

- rule governance
- campaign governance
- dispute governance
- observability

Regras de API que nao devem ser quebradas

- nenhum endpoint financeiro sem `Idempotency-Key`
- nenhum endpoint de create critico sem `X-Correlation-Id`
- nenhum endpoint de incentivo sem venda validada
- nenhum endpoint de seguranca deve retornar biometria bruta
- nenhum endpoint de parceiro deve ler tabela fora do dominio exposto pelo service owner

\n\n\n# Fonte: MANUAL_ONLINE\04_ROADMAP_IMPLEMENTACAO_SPRINTS.md\n\n\n# Roadmap de Implementacao por Sprint

Este roadmap assume sprints de 2 semanas.

Ele parte do estado atual da arvore Valley, onde a camada de banco ja materializou migrations `001-011` no PostgreSQL e `mongo-001-003` no MongoDB.

Estado Atual

Ja existe base suficiente para iniciar implementacao de produto sobre contratos reais:

- identidade, wallets e ledger
- control plane e rule definitions
- commerce, estoque, marketplace e storefront
- GOLD, Pepitas e validacao de venda
- delivery, mobility e security
- tech, legal e docs
- IA, social, telemetria, dispatch e agenda

O que ainda falta e transformar schema em APIs, workers, automacoes e produtos operacionais.

Fila Prioritaria De Release

Antes de ampliar o escopo para os 47 modulos em superficie de produto, o release funcional precisa deixar cinco modulos em estado verde operacional:

- `PAY` como no transacional: wallet bootstrap, authorize, settle, refund e ledger imutavel.
- `MARKETPLACE` como motor de oferta: storefront, zona, listing, price-check e publish gate competitivo.
- `SERVICES` como contratacao de trabalho real: provider profile, catalogo, booking, trilha append-only e integracao juridica/financeira.
- `SOCIAL` como aquisicao e atribuicao: upload, feed, metricas, campanha e comissao ligada a order/transaction.
- `CHAT` como retencao assistida: conversa, persona, memoria contextual e follow-up sem duplicar estado financeiro.

Racional:

- `PAY` bloqueia qualquer fluxo de receita, split, refund e conciliacao.
- `MARKETPLACE` desbloqueia a principal jornada de browse para order.
- `SERVICES` fecha a venda de trabalho e reputacao profissional, que e um dos vetores de monetizacao mais diretos.
- `SOCIAL` acelera aquisicao e atribuicao sem exigir o rollout imediato de todos os modulos de media.
- `CHAT` fecha a camada de retencao e operacao assistida conectando agenda, advisor e contexto do usuario.

Fila seguinte recomendada depois desse gate:

- `17-NEWS-PODCAST`
- `18-ADS`
- `19-INFLUENCERS`
- `38-AGENDA`
- `39-ADVISOR`

Sprint 0 - Baseline Tecnica

Objetivo:

- consolidar contrato de servicos, naming de eventos e padrao de API

Entrega:

- arquitetura tecnica aprovada
- mapa de ownership por servico
- padrao `X-Correlation-Id`
- padrao `Idempotency-Key`
- envelope padrao de eventos

Critério de saída:

- nenhum time novo cria endpoint ou evento fora do contrato-base

Sprint 1 - Identity + Pay Bootstrap

Objetivo:

- colocar de pe onboarding, KYC e wallet bootstrap

Entrega:

- `identity-access-service`
- `wallet-ledger-service`
- endpoints de cadastro, perfil PJ/Rider e bootstrap de wallet
- admin basico para consulta de identidade e status KYC

Banco tocado:

- `users`, `pj\_profiles`, `rider\_profiles`, `wallets`, `led\_cards`

Eventos:

- `identity.user.created`
- `identity.user.kyc\_updated`
- `wallet.created`

Critério de saída:

- usuario consegue nascer, validar identidade e obter carteira

Sprint 2 - Merchant, Catalogo e Listing Competitivo

Objetivo:

- colocar merchant, storefront, item, listing e price-check no ar

Entrega:

- `catalog-commerce-service`
- CRUD de storefront e service zones
- CRUD de item e listing

- worker de `price-check`
- snapshots de concorrência e auto-pausa do listing

Banco tocado:

- `suppliers`
- `inventory\_`*
- `marketplace\_listings`
- `merchant\_storefronts`
- `merchant\_service\_zones`
- `marketplace\_listing\_controls`
- `marketplace\_competitor\_snapshots`

Eventos:

- `catalog.listing.created`
- `catalog.listing.price\_checked`
- `catalog.listing.activated`
- `catalog.listing.auto\_paused`

Critério de saída:

- merchant publica anúncio somente quando a regra de competitividade liberar

Sprint 3 - Orders, Checkout e Receipts

Objetivo:

- transformar listing em pedido pago e rastreável

Entrega:

- `orders-orchestration-service`
- checkout end-to-end
- authorize/settle financeiro
- receipts e documentos básicos

Banco tocado:

- `orders`
- `transactions`

- `document\_records`
- `docs\_receipts`

Eventos:

- `order.placed`
- `ledger.transaction.authorized`
- `ledger.transaction.settled`
- `docs.receipt.generated`

Criterio de saida:

- usuario fecha compra e o sistema gera evidencia de pagamento

Sprint 4 - Rule Runtime, GOLD e Pepitas

Objetivo:

- ligar crescimento, incentivo e governanca de regra ao fluxo de venda

Entrega:

- `rules-growth-service`
- CRUD admin de binding runtime
- create/fund de campanha GOLD
- validacao de venda marketplace e fisica
- grant e redeem de Pepitas

Banco tocado:

- `rule\_runtime\_bindings`
- `rule\_execution\_events`
- `gold\_campaigns`
- `sale\_validation\_events`
- `gold\_campaign\_events`
- `pepita\_accounts`
- `pepita\_ledger`

Eventos:

- `rules.binding.activated`

- `growth.gold\_campaign.created`
- `growth.sale.validated`
- `loyalty.pepita.granted`

Criterio de saida:

- merchant compra GOLD e o usuario recebe Pepita apenas quando a venda for validada

Sprint 5 - Delivery, Dispatch e Prova de Entrega

Objetivo:

- operacionalizar entrega urbana com rider assignment e trilha de campo

Entrega:

- `delivery-dispatch-service`
- shipment API
- dispatch engine inicial
- timeline de eventos de entrega
- prova de entrega e documento associado

Banco tocado:

- `delivery\_shipments`
- `delivery\_shipment\_events`
- `delivery\_dispatch\_runs`
- `log\_tracking\_events`

Eventos:

- `delivery.shipment.created`
- `delivery.dispatch.started`
- `delivery.rider.assigned`
- `delivery.shipment.delivered`

Criterio de saida:

- pedido confirmado consegue virar shipment despachado e concluido

Sprint 6 - Mobility + Security

Objetivo:

- colocar corrida urbana e resposta a risco no mesmo backbone operacional

Entrega:

- `mobility-service`
- `security-response-service`
- create/update de trip
- checkpoints de corrida
- SOS, trusted contacts e incident workflow

Banco tocado:

- `mobility\_trips`
- `mobility\_trip\_events`
- `security\_trusted\_contacts`
- `security\_biometric\_credentials`
- `security\_incidents`
- `security\_incident\_events`
- `security\_signal\_logs`

Eventos:

- `mobility.trip.started`
- `mobility.trip.completed`
- `security.signal.opened`
- `security.incident.created`

Critério de saída:

- corrida funciona com trilha mínima e incidente consegue ser aberto e encerrado

Sprint 7 - Tech, Legal e Docs Operacionais

Objetivo:

- habilitar parceiros externos, webhooks, contratos e disputa

Entrega:

- `tech-platform-service`
- `legal-docs-service`
- onboarding de API client
- assinatura de contrato
- webhook subscription e audit trail
- fallback PIN e documento de prova

Banco tocado:

- `tech\_`*
- `legal\_`*
- `document\_records`

Eventos:

- `tech.client.created`
- `tech.webhook.subscribed`
- `legal.contract.signed`
- `docs.document.generated`

Criterio de saida:

- parceiro consegue integrar e contrato passa a ter trilha assinada

Sprint 8 - Social, Media e Attribution

Objetivo:

- conectar social/influencers com growth e marketplace

Entrega:

- `social-media-service`
- publicacao de videos
- ingestion de metricas
- ligacao entre creator, campanha e order/commission

Banco tocado:

- `social\_videos`
- `influencer\_metrics`

- `creator\_uploads`
- `affiliate\_referrals`

Eventos:

- `social.video.published`
- `social.campaign.metric\_aggregated`
- `growth.sale.validated`

Criterio de saida:

- campanha social consegue medir impressao, clique, conversao e comissao

Sprint 9 - AI, Agenda e Follow-up de Valor

Objetivo:

- colocar retencao assistida por IA em cima da operacao ja funcional

Entrega:

- `ai-agenda-service`
- agenda inteligente
- memoria contextual
- follow-up de compra, saude, pagamento e seguranca

Banco tocado:

- `ai\_memory`
- `agenda\_items`
- `advisor\_insights`
- `chat\_`
- `financial\_goals`

Eventos:

- `ai.memory.created`
- `agenda.item.scheduled`
- `advisor.insight.generated`

Criterio de saida:

- usuario recebe lembrete, follow-up e recomendacao ligados ao contexto real do ecossistema

Sprint 10 - Hardening, SRE e Cutover

Objetivo:

- deixar o backbone pronto para trafego real e suporte operacional

Entrega:

- tracing distribuido
- retry e dead-letter por topico
- rate limit por client e modulo
- replay controlado de outbox
- testes de carga e caos
- dashboards executivos e operacionais

Criterio de saida:

- o sistema aguenta transacao, dispatch, trilha append-only e eventos sem perder rastreabilidade

Sequencia Recomendada de Squads

Squad 1:

- core identity/pay/admin

Squad 2:

- marketplace/growth/merchant

Squad 3:

- delivery/mobility/security

Squad 4:

- tech/legal/docs/social/ai

Para release reduzido com um time enxuto:

- priorizar `PAY` + `MARKETPLACE` + `SERVICES` como Wave 1

- ativar `SOCIAL` + `CHAT` como Wave 2
- puxar `NEWS-PODCAST`, `ADS`, `INFLUENCERS`, `AGENDA` e `ADVISOR` apenas depois da prova de receita e retencao

Se o time for menor:

- executar sprints 1 a 4 primeiro
- depois 5 e 6
- depois 7 a 10

O que fica fora do roadmap imediato

- refatorar para `database-per-service`
- reescrever o banco para outro modelo
- colocar todos os 47 modulos em producao de uma vez
- criar front-end completo de todos os modulos antes de validar as jornadas centrais

Definicao de pronto global

Uma sprint so deve ser considerada pronta quando fechar os quatro planos ao mesmo tempo:

- schema e integridade
- API e contrato de erro
- eventos emitidos/consumidos
- observabilidade e evidencia operacional

\n\n\n# Fonte: MANUAL_ONLINE\DECISOES_IMPLANTACAO_V47.md\n\n\n# Decisoes de Implantacao v47 - Valley

Este documento registra como os PDFs v47 foram analisados e adaptados para esta arvore Valley.

A decisao principal continua sendo `core-first`: `users` e `wallets` sao o centro, e todos os novos objetos viaveis apontam para `users.user\_id`.

Fontes Verificadas

Foram verificados os PDFs locais do `Meu Drive`:

- `Esquema Consolidado do Valley Omniverse v47.pdf`
- `Valley Omniverse v47 - Esquema de Banco de Dados.pdf`
- `Valley Omniverse v47 - Esquema de Banco de Dados 2.pdf`

- `Valley Omniverse - Mapeamento de Modulos (v47).pdf`
- `Indice Oficial Valley Omniverse.pdf`
- `Painel Web Admin - Especificacao Consolidada (v47).pdf`
- `Valley Omniverse - Papeis de Gemini, Code Assist e Codex e Integracao de IA.pdf`
- `Valley Omniverse - Regras Consolidadas do Codex.pdf`
- `Valley Omniverse - Regras Consolidadas para o Gemini Code Assist (v47).pdf`

Implantado

`database/postgres/004\_v47\_control\_plane\_modules\_rules.sql` implanta o plano de controle v47.

Ele cria `module\_catalog`, `admin\_users`, `admin\_permissions`, `business\_rule\_definitions`, `business\_rule\_versions`, `business\_rule\_audit`, `gamification\_campaigns`, `points\_ledger`, `observability\_incidents`, `document\_records` e `admin\_action\_audit`.

Ele tambem popula os 41 modulos do mapeamento oficial v47 e registra regras base: Stock margem 50%, Mobility 10%, Food 15%, afiliados 5%, Ring-Fence Financeiro e Consentimento de Execucao Advisor.

`database/postgres/005\_v47\_domain\_tables\_core\_first.sql` implanta tabelas dominio viaveis do PDF de banco.

Ele cria `advisor\_insights`, `financial\_goals`, `teletherapy\_sessions`, `creator\_uploads`, `chat\_conversations`, `chat\_messages`, `business\_invoices`, `business\_payrolls`, `plug\_transactions`, `affiliate\_referrals` e `docs\_receipts`.

`database/postgres/006\_v47\_column\_comments\_ptbr.sql` documenta as novas tabelas em `COMMENT ON`.

Ele explica colunas, triggers e integracoes em portugues simples, mantendo termos tecnicos como `foreign key`, `JSONB`, `append-only`, `ledger`, `RBAC`, `ABAC` e `Web Admin`.

`config/modules\_v47.json` foi criado como registry canonico dos 47 modulos reais do Esquema Consolidado.

`scripts/valley\_module\_automation.py` foi criado como motor local de automacao de implantacao, desenvolvimento e evolucao.

`modules/` foi materializado com uma pasta por modulo, cada uma contendo `README.md` e `STATUS.md`.

Cada modulo agora tambem possui `CONTRACT.md`, que define fronteira operacional, politica de dados, integracoes e regras de evolucao.

`output/module-roadmap/VALLEY\_MODULE\_ROADMAP.md` foi criado como roadmap automatizado.

`output/module-roadmap/VALLEY\_MODULE\_CONTRACTS.md` foi criado como matriz consolidada dos contratos operacionais.

`database/postgres/007\_v47\_module\_delivery\_automation.sql` foi gerado a partir do registry para persistir `module\_delivery\_registry`, `module\_evolution\_backlog` e `module\_automation\_runs`.

`database/postgres/008\_v47\_foundation\_commerce\_operations.sql` foi criado para materializar contratos especificos de REPLY, STOCK, WMS e MARKETPLACE.

Esse script cria fornecedores, armazens, itens, lotes, movimentos de estoque append-only, listings, compras, linhas de compra, ordens de servico e contagens fisicas.

`database/mongodb/002\_v47\_log\_iot\_foundation.mongo.js` foi criado para materializar contratos especificos de LOG, IoT e snapshots WMS.

Esse script cria validators e indices para tracking logistico, devices, eventos de sensores e snapshots de armazem.

`database/postgres/009\_v47\_tech\_legal\_platform\_contracts.sql` foi criado para materializar contratos especificos de TECH e LEGAL.

Esse script cria API clients, credenciais por hash, conectores, webhooks, tentativas append-only, contratos, partes, assinaturas append-only, disputas, eventos juridicos append-only e fallback PIN por hash.

`database/postgres/010\_v47\_rule\_growth\_marketplace\_runtime.sql` foi criado para materializar a camada de Rule Engine runtime, growth e operacao comercial.

Esse script cria bindings e execucoes append-only do Rule Engine, storefronts, zonas de atendimento, controles de competitividade, snapshots de concorrência, contas/ledger de Pepitas, campanhas GOLD e validacao de venda marketplace/fisica.

`database/postgres/011\_v47\_city\_ops\_delivery\_mobility\_security.sql` foi criado para materializar a camada relacional de campo e seguranca.

Esse script cria shipments, eventos append-only de entrega, trips, checkpoints append-only, contatos confiaveis, biometria por hash, incidentes e eventos append-only de seguranca.

`database/mongodb/003\_v47\_field\_ops\_security\_agenda.mongo.js` foi criado para materializar a camada NoSQL de dispatch, frota, sinais de seguranca e agenda inteligente.

Esse script cria validators e indices para dispatch de entrega, cadastro de frota, eventos de manutencao, sinais de seguranca e agenda da Helena.

`database/migrations.json` foi criado como manifesto oficial da ordem de migrations.

`scripts/valley\_db\_orchestrator.py` foi criado para validar ambiente, manifesto, SQL, MongoDB, registry e aplicar migrations quando houver banco disponivel.

`docker-compose.yml` foi criado para Postgres e Mongo locais de validacao runtime.

`database/README.md`, `MANUAL\_ONLINE/OPERACAO\_AUTONOMA.md` e `output/deployment/VALLEY\_DEPLOYMENT\_STATUS.md` foram criados para registrar a esteira operacional.

Adaptado

Os schemas legados dos PDFs (`platform`, `memory`, `fintech`, `rules`, `gamification`, `observability`, `docs`, `chat`, `business`, `plug`, `affiliates`) foram achatados para `public`.

As FKs antigas para `platform.users(user\_id)` foram adaptadas para `public.users(user\_id)`.

O índice de 41 módulos foi preservado como visão estratégica, mas a automação operacional usa os 47 módulos do Esquema Consolidado.

Conceitos como `LOYALTY`, `API`, `COMMAND\_CENTER`, `ADS\_INTELLIGENCE`, `CLOUD`, `CONNECT` e `CREATOR` foram tratados como capacidades transversais quando não aparecem como módulo numerado no esquema de 47.

Os valores monetários em BRL foram padronizados como `DECIMAL(18,4)`.

Os valores tokenizados NEX continuam como `DECIMAL(18,8)`.

Logs financeiros, pontos, documentos, auditoria de regras, ações admin, Plug, referrals e receipts foram tratados como append-only quando representam auditoria ou dinheiro.

O desenvolvimento dos 47 módulos foi adaptado para passar por contrato operacional antes de schema específico.

Essa camada reduz retrabalho porque fixa `data\_home`, dependências, integrações e regras de evolução antes da implementação.

Os módulos REPLY, STOCK, LOG, WMS e MARKETPLACE foram promovidos para `implemented\_partial` no registry, porque agora possuem contrato operacional e primeira camada de schema.

LOG e IoT continuam fora do PostgreSQL para eventos volumosos; apenas referências UUID e snapshots relacionais ficam conectados ao core.

Os módulos TECH e LEGAL foram promovidos para `implemented\_partial` no registry, porque agora possuem contrato operacional e primeira camada de schema.

Credenciais de API, webhook secrets, IPs, device fingerprints e fallback PIN foram adaptados para hash, descartando armazenamento de segredo bruto por segurança.

O módulo ADS foi promovido para `implemented\_partial` no registry, porque agora possui primeira camada operacional de campanhas GOLD e eventos de conversão.

O cap de Pepita foi adaptado para enforcement relacional em `sale\_validation\_events` e `pepita\_ledger`, descartando a ideia inviável de cashback livre sem referência de margem.

O Rule Engine não foi recriado do zero; a implementação reutiliza `business\_rule\_definitions` e `business\_rule\_versions` e adiciona apenas o runtime auditável necessário.

Os módulos DELIVERY, FLEET, SECURITY e AGENDA foram promovidos para `implemented\_partial` no registry, porque agora possuem primeira camada de schema específico.

Biometria foi adaptada para persistência somente por hash e metadados, descartando armazenamento de template bruto no banco.

Os incidentes de segurança foram adaptados para modelo híbrido: contrato relacional no PostgreSQL e sinais volumosos no MongoDB.

Descartado

Não foi implantado multi-schema legado, porque conflita com a diretriz desta worktree.

Não foram implantados comandos automáticos de `git commit`, `git push`, deploy ou exclusão de arquivos, porque isso é perigoso em uma árvore local sem repositório Git validado.

Não foram implantados logs brutos de observabilidade em Postgres, porque alto volume deve ir para MongoDB, Elastic, Loki, ClickHouse ou outro backend especializado.

Não foram implantadas credenciais, tokens, gateways reais ou integrações externas, porque os PDFs descrevem intenção de produto, não contratos seguros de ambiente.

Não foi criada tabela duplicada de `platform.users`, porque `public.users` já é o no central absoluto.

Não foi criada compatibilidade direta com `uuid\_generate\_v4()`, porque esta base usa `pgcrypto` e `gen\_random\_uuid()`.

Não foi executada aplicação runtime em banco real neste ciclo, porque o ambiente atual não expõe `psql`, `mongosh`, `DATABASE\_URL` nem `MONGODB\_URI`.

Não foi forçado Docker daemon quando ele não estava comprovadamente pronto, porque a esteira segura deve registrar pendência em vez de travar o ciclo autônomo.

Risco Residual

Os scripts SQL ainda precisam ser executados em um PostgreSQL real para validação runtime.

Este ambiente não possui `psql`, então a validação feita aqui cobre estrutura textual, sintaxe JavaScript do Mongo e geração de documentação.

Quando `psql` estiver disponivel, a ordem recomendada e executar o manifesto `database/migrations.json`.

O script `003` pode rodar antes de `004/005`, mas `006` precisa vir depois das novas tabelas v47.

Depois desta atualizacao, a ordem recomendada passou a ser `001`, `002`, `004`, `005`, `003`, `006` e `007`.

Depois da evolucao foundation, a ordem PostgreSQL passou a incluir `008` apos `007`.

Depois da evolucao TECH/LEGAL, a ordem PostgreSQL passou a incluir `009` apos `008`.

Depois da evolucao de Rule Engine, Pepitas e Ads, a ordem PostgreSQL passou a incluir `010` apos `009`.

Depois da evolucao city ops e seguranca, a ordem PostgreSQL passou a incluir `011` apos `010`.

No MongoDB, a ordem passou a ser `mongo-001`, depois `mongo-002` e por fim `mongo-003`.

O relatorio operacional mais recente fica em `output/deployment/VALLEY\_DEPLOYMENT\_STATUS.md`.

\n\n\n# Fonte: MANUAL_ONLINE\NORMA_UNIFICADA_V47.md\n\n\n# Norma Unificada V47

Esta norma consolida os PDFs oficiais do Valley Omniverse e resolve conflitos de diretriz pela regra que mais protege estabilidade, seguranca, continuidade de entrega, privacidade e coerencia arquitetural.

Hierarquia De Decisao

1. Seguranca, privacidade, integridade do repositorio e conformidade legal vencem autonomia, velocidade ou conveniencia.
2. Contratos canonicos de dados vencem preferencias locais de implementacao.
3. Continuidade de entrega com estado real reportado vence simulacao de saude ou promessa otimista.
4. Automacao deve existir, mas nunca para vaziar segredo, misturar dominios sensiveis ou corromper trilha auditavel.
5. Quando dois caminhos forem possiveis, torna-se norma o que maximiza manutenibilidade documentada e reduz retrabalho.

Fontes Canonicas

1. `Valley Omniverse — Regras Consolidadas do Codex.pdf`
2. `Valley Omniverse – Regras Consolidadas para o Gemini Code Assist (v47).pdf`
3. `Painel Web Admin — Especificação Consolidada (v47).pdf`
4. `Esquema Consolidado do Valley Omniverse v47.pdf`
5. `Valley Omniverse v47 – Esquema de Banco de Dados.pdf`
6. `Valley Omniverse v47 – Esquema de Banco de Dados 2.pdf`

7. `Valley Omniverse – Mapeamento de Módulos (v47).pdf`

8. `Índice Oficial Valley Omniverse.pdf`

9. `Valley Omniverse – Papéis de Gemini, Code Assist e Codex e Integração de IA.pdf`

Regras Tornadas Norma

1. Regra Suprema De Conflito

Quando houver contradicao entre autonomia irrestrita e segurança operacional, vale segurança operacional com entrega continua.

Quando houver contradicao entre velocidade e clareza arquitetural, vale clareza arquitetural documentada.

Quando houver contradicao entre automacao hostil ao ambiente e automacao controlada via esteira, vale a esteira controlada.

2. Dados E Limites

`users.user\_id` e `wallets` continuam como nucleo absoluto.

Dados de saúde mental, score financeiro e informacoes sensiveis ficam em ring-fence; admin nao acessa conteudo bruto, apenas metadado minimo e trilha autorizada.

Ledgers financeiros, equity, claims, eventos juridicos e auditorias operacionais permanecem append-only.

3. Painel Admin

Todo modulo deve aparecer no painel admin com:

- identidade tecnica;
- status e checklist;
- docs de operacao;
- dependencias e integracoes;
- acoes administrativas;
- trilha de automacao.

O painel admin deixa de ser um artefato manual e passa a ser gerado automaticamente pela esteira.

4. Tooling E Execucao

O runtime canonico de banco local e o `builder` no Docker Compose. Isso vira norma porque evita dependencia fraca de `psql` e `mongosh` no host.

Host tooling continua util para edicao e observabilidade, mas a execucao de release e aplicacao de banco prioriza o Compose.

Toda falha de Docker, Compose, bridge, token ou extensao deve ser reportada como bloqueio real; nao pode ser mascarada como saudavel.

5. IA E Orquestracao

Codex decide e integra.

Gemini sugere, analisa e aponta melhorias.

Code Assist opera como executor assistivo de codigo e ajustes.

Sugestao de IA so vira norma local quando respeita esta hierarquia:

1. seguranca e privacidade;
2. contratos de dados e arquitetura core-first;
3. estabilidade do repositorio e da esteira;
4. utilidade operacional para admin e times internos.

6. Builds E Producao

Builds de producao devem ser minimos, sem ferramentas de debug ou coletores de log de desenvolvimento.

Segredos nao entram em settings, JSON de repo, docs ou artefatos de painel.

Toda automacao nova precisa deixar trilha visivel em `MANUAL\_ONLINE`, `config/` ou `tmp/`.

Aplicacao Pratica No Workspace

- O painel admin passa a ser refletido por `admin/valley\_admin\_data.js`.
- O builder Compose passa a ser o executor padrao de release do banco.
- O workspace passa a ter manifestos formais para extensoes, ferramentas, perfis de terminal e squad de agentes.
- A configuracao local deve privilegiar comandos nao interativos, caminhos absolutos e estado documentado.

\n\n\n# Fonte: MANUAL_ONLINE\OPERACAO_AUTONOMA.md\n\n\n# Operacao Autonoma - Valley

Este documento define a esteira autonoma atual do projeto Valley.

Ela foi criada para desenvolver, validar, implantar e evoluir os 47 modulos sem depender de confirmacoes manuais repetitivas.

Principio

A automacao deve fazer a melhor escolha segura dentro da arvore Valley.

Ela nao deve executar acoes destrutivas, apagar dados, vazsar segredo, criar credenciais reais ou fazer `git push` automatico sem contexto de repositorio validado.

Componentes

`config/modules\_v47.json` e o registry dos 47 modulos.

`scripts/valley\_module\_automation.py` gera documentacao por modulo, roadmap e migration SQL de delivery registry.

Ele tambem gera `CONTRACT.md` por modulo e a matriz
`output/module-roadmap/VALLEY\_MODULE\_CONTRACTS.md`.

`database/migrations.json` define a ordem oficial das migrations PostgreSQL e MongoDB.

`scripts/valley\_db\_orchestrator.py` valida ambiente, manifesto, artefatos dos modulos, SQL, Mongo script, registry e pode aplicar migrations quando banco estiver disponivel.

`docker-compose.yml` define Postgres e Mongo locais para validacao de implantacao.

`output/deployment/VALLEY\_DEPLOYMENT\_STATUS.md` e o relatorio operacional gerado pelo orquestrador.

`database/postgres/008\_v47\_foundation\_commerce\_operations.sql` e a primeira entrega de schema especifico dos modulos foundation de comercio/ERP.

`database/mongodb/002\_v47\_log\_iot\_foundation.mongo.js` e a primeira entrega NoSQL especifica de LOG, IoT e WMS sensor snapshots.

`database/postgres/009\_v47\_tech\_legal\_platform\_contracts.sql` e a primeira entrega de schema especifico dos modulos foundation TECH e LEGAL.

`database/postgres/010\_v47\_rule\_growth\_marketplace\_runtime.sql` e a entrega que fecha runtime de regras, growth, Pepitas, GOLD e validacao comercial/fisica.

`database/postgres/011\_v47\_city\_ops\_delivery\_mobility\_security.sql` e a entrega que aprofunda operacao de campo para Delivery, Mobility e Security.

`database/mongodb/003\_v47\_field\_ops\_security\_agenda.mongo.js` e a entrega NoSQL que fecha dispatch, frota, sinais de segurança e agenda inteligente.

Ciclo Natural

1. Atualizar ou revisar o registry dos 47 módulos.
2. Rodar `python scripts/valley\_module\_automation.py validate`.
3. Rodar `python scripts/valley\_module\_automation.py sync`.
4. Rodar `python scripts/valley\_module\_automation.py sql`.
5. Rodar `python scripts/valley\_db\_orchestrator.py check`.
6. Rodar `python scripts/valley\_db\_orchestrator.py report`.
7. Atualizar o Manual Online.
8. Rodar `python scripts/generate\_manual\_pdf.py`.

`sync` deve manter `README.md`, `STATUS.md`, `CONTRACT.md`, `modules/INDEX.md`, roadmap e matriz de contratos alinhados.

`contracts` pode ser usado quando a mudança for apenas na fronteira operacional dos módulos.

Gate Mínimo De Release Funcional

Nenhum release do backbone Valley deve ser tratado como pronto se os módulos abaixo não estiverem com evidência técnica fechada em `STATUS.md`:

- `modules/08-pay/STATUS.md`
- `modules/07-marketplace/STATUS.md`
- `modules/10-services/STATUS.md`
- `modules/20-social/STATUS.md`
- `modules/46-chat/STATUS.md`

Sentido operacional de cada gate:

- `PAY`: prova que identidade, wallet e ledger aguentam a jornada financeira principal.
- `MARKETPLACE`: prova que a oferta comercial consegue nascer, ser precificada e ser publicada com governança.
- `SERVICES`: prova que a plataforma vende trabalho/agenda real sem romper financeiro nem contrato.
- `SOCIAL`: prova que aquisição e atribuição funcionam fora do canal local puro.
- `CHAT`: prova que o follow-up e a operação assistida conseguem reter usuário no ecossistema.

Sequência recomendada do gate:

1. fechar status e testes de `PAY`
2. fechar status e testes de `MARKETPLACE`
3. fechar status e testes de `SERVICES`
4. fechar status e testes de `SOCIAL`
5. fechar status e testes de `CHAT`

Fila natural logo apos esse gate:

- `17-NEWS-PODCAST` para superficie publica e testes externos de conteudo
- `18-ADS` e `19-INFLUENCERS` para escalar growth
- `38-AGENDA` e `39-ADVISOR` para retencao inteligente sobre a base ja operacional

Implantacao Local Com Docker

Quando o Docker daemon estiver disponivel, o fluxo local e:

```
python scripts/valley_db_orchestrator.py apply-compose
python scripts/valley_db_orchestrator.py report
python scripts/generate_manual_pdf.py
```

Esse fluxo sobe Postgres e Mongo, espera readiness real, executa o service `builder` do Docker Compose, aplica migrations e atualiza a evidencia operacional.

Quando quiser rodar o worker explicitamente:

```
docker compose --profile builder run --rm --build builder
```

Implantacao Sem Docker

Quando houver banco externo e `.env`/`.env.example` estiverem na raiz:

```
python scripts/valley_db_orchestrator.py apply-postgres
python scripts/valley_db_orchestrator.py apply-mongo
```

Quando precisar sobrescrever manualmente as conexoes:

```
DATABASE_URL=postgresql://user:pass@host:5432/db python scripts/valley_db_orchestrator.py
apply-postgres
MONGODB_URI=mongodb://host:27017/valley python scripts/valley_db_orchestrator.py apply-mongo
```

No PowerShell:

```
$env:DATABASE_URL='postgresql://user:pass@host:5432/db'; python scripts/valley_db_orchestrator.py
apply-postgres
$env:MONGODB_URI='mongodb://host:27017/valley'; python scripts/valley_db_orchestrator.py apply-mongo
```


Politica De Descarte

Ideias inviáveis devem ser descartadas quando:

- conflitam com `users` e `wallets` como núcleo absoluto;
- exigem segredo real que não existe no ambiente;
- armazenam segredo bruto, API key, webhook secret, IP bruto, fingerprint bruto ou fallback PIN em texto claro;
- liberam cashback/Pepita sem cap relacional ligado a margem ou validação de venda;
- duplicam schema legado como `platform.users`;
- tentam mover log bruto de alto volume para PostgreSQL;
- fazem deploy, commit, push ou delete automático sem trilha segura.

Estado Atual

O ambiente local tem Docker Compose, mas não tem `psql`, `mongosh` nem `make` no PATH.

Por isso, a validação principal agora é estática e documental quando o Docker daemon não responde.

Quando Docker daemon e imagens estiverem prontos, `apply-compose` passa a ser o caminho natural de validação runtime porque ele mesmo sobe o ambiente antes de aplicar.

O último relatório gerado deve ser lido em `output/deployment/VALLEY\_DEPLOYMENT\_STATUS.md`, porque o total muda quando novas migrations entram no manifesto.

O painel admin fica em `admin/index.html` e consome `admin/valley\_admin\_data.js`, gerado automaticamente pela esteira de módulos e pelo report do banco.